Architecture Determines Optimization: Deriving Weight Updates from Network Topology

Cameron L. Bass
Independent Researcher
cameronlbass@gmail.com

Abstract

We derive neural network weight updates from first principles without assuming gradient descent or a specific loss function. Starting from the error equation $E = y - f(x)$ and linearizing with respect to the free parameters, while noting that the data x and target y are fixed observations, we obtain a linear constraint on the weight perturbations. The minimum-norm solution to this underdetermined system is algebraically identical to the standard weight gradient. Gradient descent is therefore a consequence of minimizing weight change subject to error correction.

This derivation exposes three structural facts obscured by the standard loss-based approach. First, the minimum-norm solution uses the L_2 norm to determine the update direction, but the weights within a layer are structurally independent, making the L_1 norm the appropriate measure of update magnitude; the L_1 normalization also provides scale invariance with respect to input magnitude. Second, the denominator in the minimum-norm solution averages over continuous input dimensions; a layer in a neural network is a discrete space with d_l independent outputs, each training step addresses one correction among d_l alternatives, and $O(\ln(d_l))$ effective measurements are required, yielding a per-layer step size of $1/\ln(d_l)$. Third, while the standard gradient vanishes at convergence, the Hessian also vanishes, leaving converged weights with no restoring force against perturbation, which provides a first-principles explanation for catastrophic forgetting and clarifies the role of error-based updates in maintaining instantaneous correction.

Index terms — backpropagation, catastrophic forgetting, compressed sensing, hyperparameter-free optimization, minimum-norm solution, neural network optimization

I. Introduction

Neural network training has proceeded from the same premise since Cauchy [1]: define a loss function $L(w)$, compute the gradient for each weight $\partial L/\partial w$, step in the negative gradient direction. The gradient is computed by backpropagation, wherein the chain rule is applied recursively through the network [2]. This procedure is effective. It is also, as we will show, unnecessary as a starting assumption.

We ask a different question. Given an error, the difference between a prediction and a target, what is the smallest weight change that addresses it? The answer to this question produces a weight update from the computational structure of a general neural network.

The answer, derived in Section II, is algebraically identical to the standard gradient. This means gradient descent is not a design choice, but a mathematical consequence of applying the minimum-norm solution to the error constraint of a neural network. The closest prior work is the Widrow-Hoff Delta Rule [3], which derived a similar update for single-layer adaptive filters using least mean squares. Our derivation generalizes this to arbitrary depth without assuming a specific loss function or optimization objective. The generalization is specifically through two observations: each layer's input is fixed data for that layer's constraint, allowing independent per-layer solutions (Section II-G), and the error propagates between layers through the linearized forward map (Section II-H). These two observations turn the single-layer Delta Rule into multi-layer backpropagation.

A note on the implicit role of the loss function: this derivation does not assume a specific loss function, but it does assume that the error should be reduced to zero. This goal is equivalent to steepest descent on $\|E\|^2$, so a loss function is implicit in the objective. This derivation eliminates the loss function as a starting point for computing the update. The loss function determines why we make the reduction; the minimum-norm principle determines how that reduction is achieved. Our contribution is that gradient descent is a consequence of reduction combined with the minimum-norm principle, rather than an independent design choice.

II. Derivation of the Weight Update

A. Setup

A neural network computes a prediction from data. The simplest case is a network of a single layer:

$$f = Wx + b \tag{1}$$

where x is the input data, W is the weight matrix, b is the bias vector, and f is the prediction. The error is the difference between the target and the prediction:

$$E = y - f = y - Wx - b \tag{2}$$

We wish to find weight changes that reduce E. We have three kinds of quantities:

Data (observed, fixed): the input x and the target y. These are measurements of the world. Adjusting the weights does not change the data. **Parameters** (adjustable, free): the weights W and the bias b. These are the quantities we can change. **Derived** (determined by the above): the prediction f and the error E. These are computed from the data and the parameters.

B. Linearizing the Error Change

Consider a finite weight perturbation ΔW and bias perturbation Δb. The resulting change in the prediction is:

$$\Delta f = (\Delta W)x + \Delta b \tag{3}$$

This is exact for a linear layer (no approximation required). The corresponding change in error is:

$$\Delta E = -\Delta f = -(\Delta W)x - \Delta b \tag{4}$$

We seek weight changes that satisfy:

$$(\Delta W)x + \Delta b = -\Delta E \tag{5}$$

where ΔE is the desired error change. Since our goal is to reduce the error toward zero, the natural choice is $\Delta E = -E$ to correct the entire error in one step, or $\Delta E = -\alpha E$ for some step fraction α. In either case, the direction of ΔE is the error itself. This choice is equivalent to steepest descent on $\|E\|^2$; the minimum-norm principle below determines the weight change.

C. The Augmented System

The bias Δb acts as a weight perturbation on a constant input of 1. We absorb it by defining an augmented input $\bar{x} = [x; 1]$ and augmented weight perturbation $\Delta\bar{W} = [\Delta W | \Delta b]$. Then equation (5) becomes:

$$(\Delta\bar{W})\bar{x} = -\Delta E \tag{6}$$

This is one equation with many unknowns — every entry of $\Delta\bar{W}$ is free to vary. There are more unknowns than equations, so the system is underdetermined: many different weight changes can produce the same error change.

D. The Minimum-Norm Solution

When a system of linear equations is underdetermined, the minimum-norm solution selects the solution with the smallest total magnitude. For the system $Az = c$, this is $z = A^\top(AA^\top)^{-1}c$, the Moore-Penrose pseudoinverse [4]. Among all solutions, this is the one with minimum $\|z\|^2$.

Applied to equation (6): among all $\Delta\bar{W}$ satisfying the constraint, select the one that minimizes $\|\Delta\bar{W}\|^2$ (the sum of squares of all weight changes). The solution is:

$$\Delta\bar{W} = -\Delta E \cdot \bar{x}^\top / \|\bar{x}\|^2 \tag{7}$$

Since $\bar{x} = [x; 1]$, we have $\|\bar{x}\|^2 = \|x\|^2 + 1$, and expanding:

$$\Delta W = -\Delta E \cdot x^\top / (\|x\|^2 + 1) \tag{8}$$

$$\Delta b = -\Delta E \cdot 1 / (\|x\|^2 + 1) \tag{9}$$

In component form, for a layer with multiple outputs, each with error component e_i:

$$\Delta w_{ij} = -e_i \cdot x_j / (\|x\|^2 + 1) \tag{10}$$

$$\Delta b_i = -e_i / (\|x\|^2 + 1) \tag{11}$$

The +1 in the denominator is the squared norm of the constant input ($1^2 = 1$) from the augmented bias dimension.

E. Equivalence to the Gradient

Equations (10) and (11) give the weight and bias changes for a single sample. For a batch of B samples, we average:

$$\Delta w_{ij} = -(1/B) \sum_s e_i^{(s)} \cdot x_j^{(s)} / (\|x^{(s)}\|^2 + 1) \tag{12}$$

$$\Delta b_i = -(1/B) \sum_s e_i^{(s)} / (\|x^{(s)}\|^2 + 1) \tag{13}$$

If we set aside the denominator $(\|x\|^2 + 1)$, which we will address in Section IV, the updates reduce to:

$$\Delta w_{ij} = -\text{mean}(e_i \cdot x_j) \tag{14}$$

$$\Delta b_i = -\text{mean}(e_i) \tag{15}$$

For cross-entropy loss with softmax output, the gradient of the loss with respect to the logits is $e = \text{softmax}(f) - y$: the softmax residual. This quantity serves as both the "error" in our derivation (the discrepancy between prediction and target in probability space) and the gradient of the loss with respect to the logits. The gradient with respect to the weights is $\partial L / \partial w_{ij} = e_i \cdot x_j$. Equations (14)–(15) are therefore identical to the standard gradient.

For mean squared error with $E = y - f$, the error is $e_i = y_i - f_i$ directly, and the gradient is $\partial(\frac{1}{2}\|E\|^2)/\partial w_{ij} = -e_i \cdot x_j$, also identical.

In both cases, the error component e_i in equations (10)–(15) is the current error at output i — the quantity ΔE from Section II-B with $\alpha = 1$ (correct the full error in one step). The minimum-norm solution provides the weight change; the step fraction α (equivalently, a learning rate) controls how much of that change is applied.

The gradient emerges as the minimum-norm solution to the constraint imposed by the error equation.

For linear layers, the full update with denominator (equations 10–11) is the Normalized Least Mean Squares (NLMS) rule from adaptive filtering [3]. For nonlinear layers, the linearized constraint and minimum-norm solution correspond to the Gauss-Newton method [5, 6]. The present derivation provides a geometric interpretation of these classical algorithms as minimum-perturbation error corrections.

F. Scale Invariance

The denominator $\|x\|^2 + 1$ in equations (10)–(11) provides automatic scale invariance. If the inputs are rescaled $x \to cx$, then the numerator scales by $e_i \cdot (cx_j) = c \cdot (e_i \cdot x_j)$, and the denominator scales by $\|cx\|^2 + 1 = c^2\|x\|^2 + 1$.

For large $\|x\|$ (where the +1 is negligible): the update scales as $c/c^2 = 1/c$. Larger inputs produce proportionally smaller weight changes, preventing any single large-magnitude input from dominating the update. This property is intrinsic to the minimum-norm solution and requires no additional normalization.

When L_1 normalization (Section III) replaces the denominator, scale invariance is preserved: if $x \to cx$, both the numerator $e_i \cdot x_j$ and the L_1 norm $\sum |e_i \cdot x_j|$ scale by $|c|$, and their ratio is scale-independent.

G. Layer Independence

A deep network stacks L layers:

$$a_0 = x \tag{16}$$

$$z_l = W_l \cdot a_{l-1} + b_l \tag{17}$$

$$a_l = g(z_l) \tag{18}$$

where a_l is the activation at layer l, z_l is the weighted sum, and g is the activation function. The error at the output is $E = y - a_L$ (or, for classification, derived from the softmax of z_L).

Each layer can solve its own constraint independently. Layer l sees:

- Input a_{l-1}: this was computed during the forward pass. It is data for this layer's constraint, therefore $\Delta a_{l-1} = 0$.
- Error E_l: the error signal that has arrived at this layer's output.

The same argument from Section II-B applies at every layer. The input to each layer is fixed for the purpose of that layer's weight update, so that a block-coordinate descent view for each layer solves its subproblem holding activations fixed. Each layer solves:

$$\Delta E_l = -(\Delta W_l) \cdot a_{l-1} - \Delta b_l \tag{19}$$

independently, with the same minimum-norm solution:

$$\Delta w_{ij}^{(l)} = -e_i^{(l)} \cdot a_j^{(l-1)} / (\|a_{l-1}\|^2 + 1) \tag{20}$$

$$\Delta b_i^{(l)} = -e_i^{(l)} / (\|a_{l-1}\|^2 + 1) \tag{21}$$

H. Error Propagation

After layer l solves for its weight update, the error must propagate to the layer below. Layer $l - 1$ needs to know: what error should it correct?

The forward computation at layer l is: $z_l = W_l \cdot a_{l-1} + b_l$, then $a_l = g(z_l)$. Some perturbation Δa_{l-1} to the input produces:

$$\Delta z_l = W_l \cdot \Delta a_{l-1} \tag{22}$$

$$\Delta a_l = g'(z_l) \odot \Delta z_l = g'(z_l) \odot (W_l \cdot \Delta a_{l-1}) \tag{23}$$

The error signal E_l at layer l's output must be mapped to the error at layer l's input. Inverting the linearized forward map, we find the perturbation at the input that would explain the error at the output is:

$$E_{l-1} = W_l^\top \cdot (g'(z_{l-1}) \odot E_l) \tag{24}$$

where g' is the derivative of the activation function (for ReLU: 1 where $z > 0$, 0 elsewhere) and $\odot$ denotes elementwise (Hadamard) multiplication.

This is backpropagation derived from the linearized forward map, rather than differentiation of a loss function.

I. Assumptions

The derivation used:

1. A finite weight perturbation ΔW and the resulting linear change in error $\Delta E = -(\Delta W)x - \Delta b$ (exact for linear layers, first-order for nonlinear layers).
2. Data does not change: $\Delta x = 0$ and $\Delta y = 0$.
3. The minimum-norm solution: among all solutions to the underdetermined constraint, choose the one with smallest $\|\Delta \bar{W}\|^2$.
4. The goal of reducing the error toward zero: $\Delta E = -\alpha E$.
5. The chain rule, to propagate errors through nonlinear activation functions (equation 24). For purely linear networks, this is unnecessary, as errors propagate through $W_l^\top$ directly.

III. Weight Independence and the L_1 Norm

A. Two Norms, Two Roles

The minimum-norm solution in Section II-D minimizes the L_2 (Euclidean) norm $\|\Delta \bar{W}\|^2$, and this determines the update direction and relative proportions for the weights.

The L_1 norm measures the total update magnitude for independent quantities, then normalizes the magnitudes.

B. Independence Structure

The constraint equation (6) treats all components of $\Delta \bar{W}$ as free variables to be determined simultaneously. Each weight w_{ij} connects a specific input j to a specific output i. The weight w_{ij} does not constrain w_{kl} for $(i, j) \neq (k, l)$ as these are independent parameters.

This independence is structural for a directed graph. The update for w_{ij} (equation 10) depends on the error at output i and the activation at input j. Two weights connecting from different inputs (w_{ij} and w_{ik}) depend on different activations x_j and x_k. Two weights connecting to different outputs (w_{ij} and w_{mj}) depend on different errors e_i and e_m. The updates are independent, therefore the weights are independent.

C. L_1 Normalization

While the weights are structurally independent parameters with disjoint support in the weight matrix, their updates $\Delta w_{ij} = -e_i \cdot x_j$ are correlated through shared inputs x_j and shared errors e_i. The independence is parametric (each weight occupies its own slot), not statistical (updates to weights sharing an input are correlated).

L_1 normalization is adopted for two reasons:

Exact scale invariance: if inputs are rescaled $x \to cx$, both the update numerator $e_i \cdot x_j$ and the L_1 norm $\sum |e_i \cdot x_j|$ scale by $|c|$, and their ratio is independent of c. L_2 normalization (dividing by $\|\Delta w\|$) does not provide exact scale invariance because $\|cx\| = |c| \cdot \|x\|$ but $\|cx\|^2 = c^2 \cdot \|x\|^2$.

L_1 **normalization fixes the total correction distance to** $O(1)$, which is essential for deriving the step size in Section IV. Under L_1 normalization, each update has unit magnitude regardless of batch size, error scale, or input scale. This provides the "unit distance" that, combined with a specific measurement bound, yields the step size $\eta = 1/\ln(d_l)$, which we discuss in Section IV. Without a normalization that fixes the total distance, the step size cannot be derived from the measurement count alone.

D. Cross-Layer vs Within-Layer

The independence holds within a layer. Between layers, weight updates are coupled through the error propagation (equation 24): the error at layer l depends on the weights at layer $l+1$. Per-layer L_1 normalization respects this boundary by normalizing within the independent set and does not conflate coupled quantities across layers.

IV. The Discrete Topology and $\ln(d_l)$

A. The Multi-Output Pseudoinverse

The derivation in Section II treats each output independently. We define the dimension of a layer d_l as the number of independent functions computed to determine that layer's output. For a layer with d_l outputs, the full constraint system is:

$$\Delta E = -J \cdot \Delta\theta \tag{25}$$

where J is the $d_l \times P$ Jacobian (P = total parameters in the layer) and $\Delta\theta$ is the vectorized weight perturbation. The minimum-norm solution is:

$$\Delta\theta = -J^\top (JJ^\top)^{-1} \Delta E \tag{26}$$

For a linear layer, J has a specific structure: row i of J is $\bar{x}^\top$ (the augmented input) for output i. Since all outputs share the same input $\bar{x}$, the rows of J are identical up to scaling by the error components. The matrix $JJ^\top$ is $d_l \times d_l$, with entries $(JJ^\top)_{ik} = \bar{x}^\top \bar{x} \cdot \delta_{ik} = \|\bar{x}\|^2 \cdot I$ for independent outputs. This recovers the per-output denominator $\|\bar{x}\|^2$.

However, the per-output solution treats each output as if it were the only one. When d_l outputs are corrected simultaneously through shared input weights, the question becomes: how many of the d_l corrections carry independent information about the required weight change?

The $\ln(d_l)$ bound depends on the outputs sharing the input through different weight vectors. Each input weight w_j connects to all d_l outputs, creating entanglement in which the correction for one output constrains what corrections are available for the others. When outputs do not share inputs (input dimension 1 per output, as in per-channel normalization layers), each output is an independent constraint with $d_l = 1$, and the measurement bound is trivial. The criterion for applying the $\ln(d_l)$ step size is that the layer's outputs share a common input vector of dimension greater than 1; otherwise, a single measurement is enough to correct the error.

B. Effective Independent Constraints

Each input weight w_j connects to all d_l outputs. The total correction demanded of w_j is the sum of d_l terms: $\sum_i e_i \cdot x_j$, one per output. If the errors $\{e_i\}$ were independent random variables, the effective number of independent constraints would be d_l. But the errors are not independent, as they arise from a shared prediction on a shared input, making them highly correlated.

Each weight w_{ij} connects one input j to one output i. Its update depends on one error component e_i, which is exactly 1-sparse in the d_l-dimensional output space (Section III-B). Note that the total update to input j, summed across all outputs, is not sparse. The question becomes: given d_l output nodes, where the correction signal is 1-sparse per weight, how many effective independent measurements does one training step provide?

This is the question addressed by Donoho's compressed sensing (CS) [7], which proves that for an unknown vector in $\mathbb{R}^m$ with N important coefficients, $n = O(N \cdot \log(m))$ nonadaptive measurements suffice for recovery. For $N = 1$ (each weight is an important coefficient) and $m = d_l$ (output dimensions):

$$n = O(\log(d_l)) \tag{27}$$

The error vector across d_l outputs is compressible: it belongs to an ℓ_p ball with $p < 1$ in Donoho's framework [7]. The presence of any nonzero error is itself indicative of sparsity, implying a fully converged output (zero error)

requires no correction, and its coefficient in the update is exactly zero. The error signal is the sparse residual of a network with incomplete learning.

C. Step Size from Sample Complexity

CS establishes a lower bound on the number of measurements required to recover a sparse signal. For a layer with d_l outputs where the correction is 1-sparse per weight (Section III-B), Donoho's bound [7] implies that $n = O(\ln(d_l))$ independent measurements are necessary and sufficient to resolve the weight update.

In the context of iterative training, each step provides one measurement of the error. The CS bound therefore limits the convergence horizon: no algorithm can reliably resolve the correction in fewer than $O(\ln(d_l))$ steps.

The step size η determines how much of the total correction is applied per measurement. Under L_1 normalization (Section III), the update direction has unit L_1 magnitude, meaning the total correction distance is $O(1)$. To saturate the information bound and converge in the minimum number of steps allowed by the discrete topology, the step size must distribute the total correction evenly across the required measurements:

$$\eta \cdot n \approx 1 \quad \Rightarrow \quad \eta \approx 1/n = 1/\ln(d_l) \tag{28}$$

This yields the step size as a direct consequence of three factors:

Topology: The discrete output structure requires $O(\ln(d_l))$ measurements (CS bound [7]). **Normalization**: L_1 normalization (Section III) fixes the total correction magnitude to $O(1)$. **Optimality**: The step size is chosen to achieve convergence in the information-theoretic minimum number of steps.

Donoho's result [7] uses the logarithm with unspecified base. The Johnson-Lindenstrauss lemma [8] provides a related bound using Gaussian random projections: d_l points embed in $O(\ln(d_l)/\varepsilon^2)$ dimensions. The Gaussian measure underlying the JL proof produces the natural logarithm. Other bases would change only the constant factor; the natural logarithm is the form that arises from the discrete topology.

D. The Topological Step Size

The resulting per-layer step size is:

$$\eta_l = 1/\ln(d_l) \tag{29}$$

This is the step size that matches the update magnitude to the layer's information capacity, rather than a heuristic parameter. The continuous pseudoinverse denominator $\|\bar{x}\|^2$ arises from treating all input dimensions as independent measurements. The discrete step size $1/\ln(d_l)$ arises from three derived quantities: the CS measurement bound (Section IV-B), the L_1 unit normalization (Section III), and the requirement that convergence saturate the information-theoretic minimum.

Each layer has its own d_l and its own step size $1/\ln(d_l)$. Layers with fewer outputs require fewer measurements to resolve the correction, allowing larger steps. Layers with more outputs require more measurements, necessitating smaller steps. The logarithmic scaling ensures that each step contributes exactly one measurement's worth of information toward the total error correction.

For $d_l = 1$ (a single output function), $\ln(d_l) = 0$ and the step size is undefined. This is the correct limit: a single output is fully determined by one measurement, and the step size is $\eta = 1$.

V. Restoring Forces at Convergence Vanish

A. The Gradient Vanishes

The standard gradient for cross-entropy loss with softmax output is:

$$\partial L/\partial w_{ij} = e_i \cdot x_j \tag{30}$$

where $e = \text{softmax}(f) - y$. As the network converges ($p \to 1$ for the correct class), $e \to 0$, and the gradient vanishes. Both the first and second derivatives of the loss vanish at convergence:

$$\partial L/\partial f = -(1-p) \to 0 \tag{31}$$

$$\partial^2 L/\partial f^2 = p(1-p) \to 0 \tag{32}$$

The loss landscape at convergence is flat, in that both the gradient and the Hessian vanish. This means the loss-based optimizer has no gradient to push weights back toward the converged solution if they are perturbed.

B. The Vanishing Restoring Force

When the gradient vanishes, the network cannot resist perturbations. Consider a converged weight configuration w^* for task A. Some perturbation δw (from noise, new data, or training on task B) moves the weights away from w^*. The gradient at $w^* + \delta w$ determines how strongly the network "pulls back" toward w^*:

$$\partial L/\partial w\big|_{w^*+\delta w} \approx \partial L/\partial w\big|_{w^*} + (\partial^2 L/\partial w^2)\big|_{w^*} \cdot \delta w \tag{33}$$

Since both the gradient and the Hessian vanish at w^* (equations 31–32), the restoring force is:

$$\text{restoring force} \approx 0 + 0 \cdot \delta w = 0 \tag{34}$$

The weights drift freely under perturbation. The loss eventually increases because there is no mechanism to resist the drift. The weights must drift far enough that p deviates significantly from 1 before any gradient signal reappears.

C. The Loss Responds Slowly

While the restoring force vanishes, the loss itself remains sensitive to weight changes. The loss near convergence is:

$$L = -\log(p) \approx (1-p) \tag{35}$$

A perturbation δw changes p by $\delta p \approx -p(1-p) \cdot x \cdot \delta w$. The resulting loss change is:

$$\Delta L \approx (1 - p - \delta p) - (1-p) = -\delta p = p(1-p) \cdot x \cdot \delta w \tag{36}$$

For small perturbations, this is proportional to $(1-p)$. The gradient at the new position is small:

$$|\partial L/\partial w|_{w^*+\delta w}| \approx (1 - p + |\delta p|) \cdot |x| \tag{37}$$

The ratio of loss increase to available gradient for correction:

$$\Delta L/|\partial L/\partial w| \approx p(1-p) \cdot x \cdot \delta w/((1 - p + |\delta p|) \cdot x) \to \delta w \tag{38}$$

as $p \to 1$. The loss increases in proportion to the perturbation, but the gradient available to correct it also increases, so the system is not unstable in the classical sense. The pathology is not divergent curvature but delayed response: the gradient reappears only after significant drift has occurred.

D. Catastrophic Forgetting as Delayed Response

Catastrophic forgetting follows from the vanishing restoring force:

1. After convergence on task A, the gradient for task A is near zero (equation 31).
2. Training on task B perturbs weights by δw in directions useful for task B.
3. The loss-based optimizer has no gradient signal from task A to resist this perturbation (equation 34). Task A's weights drift freely.
4. By the time task A's prediction degrades enough to produce a non-negligible gradient, the weights have moved far from w^*. The accumulated drift is catastrophic.

Elastic weight consolidation [9] compensates by estimating the Fisher Information Matrix along the training trajectory (where the Fisher is nonzero) and penalizing changes to weights that were important for task A. This provides an artificial restoring force where the loss landscape's natural restoring force has vanished.

E. The Error-Based Formulation Maintains Restoring Forces

Under the error-based formulation (Section II), the update is proportional to the error $E = y - f$:

$$\Delta w_{ij} \propto -e_i \cdot x_j \tag{39}$$

At convergence, $e \to 0$ and the update vanishes in the same way as the loss-based gradient. However, the critical difference appears under perturbation. If the weights are perturbed by δw, the error changes by:

$$\delta E = -(\delta w) \cdot x \tag{40}$$

This is a linear function of δw. The error-based update at the perturbed position is:

$$\Delta w_{ij} \propto -(e_i + \delta e_i) \cdot x_j \tag{41}$$

The correction is immediately proportional to the perturbation. There is no delay and no need for the perturbation to accumulate until p deviates significantly from 1. The error formulation provides an instantaneous restoring force proportional to δE, whereas the loss formulation requires δE to propagate through $-\log(p)$ before producing a gradient, introducing the delay identified above.

F. Limitations and Predictions

The derivation in Section II is exact for linear layers and first-order for nonlinear layers. For deep nonlinear networks, the minimum-norm solution to the linearized constraint is an approximation, identical to the approximation underlying standard backpropagation [2] and the Gauss-Newton method [5, 6].

The vanishing restoring force analysis (Section V-A–V-E) is specific to cross-entropy loss with softmax. For mean squared error (MSE), the Hessian is $\partial^2 L/\partial f^2 = 1$ (constant), which does not vanish at convergence. The restoring force persists: a perturbation δw at a converged MSE solution encounters a nonvanishing Hessian that resists the displacement. This predicts that catastrophic forgetting should be less severe under MSE loss than under cross-entropy and is empirically testable.

More generally, any loss function whose second derivative with respect to the logits vanishes at convergence will exhibit the same vanishing restoring force. Loss functions with second derivatives that remain bounded away from zero (such as MSE and Huber loss) retain restoring forces at convergence and should exhibit less catastrophic forgetting. The severity of forgetting is predicted to correlate with the rate at which $\partial^2 L/\partial f^2$ vanishes.

VI. Conclusion

We have presented four results:

The gradient is a consequence. The standard neural network weight gradient is the minimum-norm solution (Moore-Penrose pseudoinverse) to the constraint imposed by the error equation, with data held fixed. No loss function or optimization theory is required as a starting point, though a loss function is implicit in the goal of reducing the error. Backpropagation emerges as error propagation between independent layer constraints, requiring the chain rule to propagate through nonlinear activations. This generalizes the Widrow-Hoff Delta Rule [3] to arbitrary depth without assuming an optimization objective.

L_1 normalization provides scale invariance and unit distance. The minimum-norm solution uses L_2 to find the update direction. L_1 normalization provides exact scale invariance with respect to input magnitude and fixes the total correction distance to $O(1)$, which is essential for deriving the topological step size.

The step size is topological. The weight update at each layer addresses a 1-sparse correction among d_l independent outputs. Compressed sensing [7] requires $O(\ln(d_l))$ measurements to resolve this. L_1 normalization fixes the total correction distance to $O(1)$. The step size $\eta = 1/\ln(d_l)$ follows: it is the distance divided by the measurement count, achieving convergence in the information-theoretic minimum number of steps. The base of the logarithm is naturally produced by the Johnson-Lindenstrauss lemma [8].

Converged weights lack restoring forces. At convergence, both the gradient and the Hessian of the cross-entropy loss vanish. The loss landscape is flat, meaning there is no gradient signal to resist weight perturbations. Catastrophic forgetting follows: training on a new task perturbs weights freely, with no restoring force from the old task until significant degradation has occurred. EWC [9] compensates by providing an artificial restoring force based on the Fisher Information. The error-based formulation (Section II) provides an instantaneous restoring force proportional to the perturbation-induced error change, without the delayed response inherent in the loss formulation.

These results reinterpret existing practice: gradient descent is not a design choice but a structural consequence, the learning rate is a constant determined by the topology, and the known pathologies of training (catastrophic forgetting, sensitivity to learning rates near convergence) trace to the second-order geometry of the loss landscape.

Remark. For a single-layer network, the above results connect CS to the Universal Approximation Theorem (UAT) [10]. The UAT guarantees that d output nodes suffice to approximate any continuous function. The minimum-norm principle (Section II) gives the direction of the weight update. Section IV gives the step size: $1/\ln(d)$. Together, these establish that a single-layer network can represent any continuous function (UAT), that the minimum-change weight correction is the gradient (Section II), and that the convergence rate per step is $1/\ln(d)$ (Section IV). The existence of the solution, the direction toward it, and the rate of approach are determined by three theorems and one observation that $\Delta x = 0$.

VII. References

[1] A. Cauchy, "Methode generale pour la resolution des systemes d'equations simultanees," *Comptes Rendus de l'Academie des Sciences*, vol. 25, pp. 536–538, 1847.

[2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.

[3] B. Widrow and M. E. Hoff, "Adaptive switching circuits," in *IRE WESCON Convention Record*, pp. 96–104, 1960.

[4] A. Ben-Israel and T. N. E. Greville, *Generalized Inverses: Theory and Applications*, 2nd ed. New York: Springer, 2003.

[5] S. Amari, "Natural gradient works efficiently in learning," *Neural Computation*, vol. 10, no. 2, pp. 251–276, 1998.

[6] J. Martens, "New insights and perspectives on the natural gradient method," *Journal of Machine Learning Research*, vol. 21, no. 146, pp. 1–76, 2020.

[7] D. L. Donoho, “Compressed sensing,” *IEEE Transactions on Information Theory*, vol. 52, no. 4, pp. 1289–1306, 2006.

[8] W. B. Johnson and J. Lindenstrauss, “Extensions of Lipschitz mappings into a Hilbert space,” *Contemporary Mathematics*, vol. 26, pp. 189–206, 1984.

[9] J. Kirkpatrick et al., “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[10] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals, and Systems*, vol. 2, no. 4, pp. 303–314, 1989.